

ZONE 2: READY - Pre-Event Validation & Readiness

Technical Specifications for eVenu360 MVP

Overview

Temporal Frame: T-7 days → T-0 (Event Start)
Primary Function: Validate operational readiness and ensure all systems meet go-live criteria
Key Outputs: Readiness scores, validation reports, go/no-go decisions, staff deployment plans

1. DATABASE SCHEMA

Core Tables

typescript

```
TypeScript
// =====
// READINESS VALIDATION TABLES
// =====

interface ReadinessChecklist {
  checklist_id: string;           // PK: uuid
  venue_id: string;              // FK: venues.venue_id
  event_id: string;              // FK: events.event_id
  checklist_type: 'safety' | 'technical' | 'operational' |
'compliance' | 'staffing';
  temporal_phase: 'T-7' | 'T-3' | 'T-1' | 'T-0';
  total_items: number;
  completed_items: number;
  completion_percentage: number;
```

```

    status: 'not_started' | 'in_progress' | 'completed' | 'failed'
| 'waived';
    created_at: timestamp;
    updated_at: timestamp;
    due_by: timestamp;
    assigned_to: string;           // FK: staff.staff_id
    metadata: {
        priority: 'critical' | 'high' | 'medium' | 'low';
        dependencies: string[];    // Other checklist_ids
        automated: boolean;
    };
}

```

```

interface ChecklistItem {
    item_id: string;              // PK: uuid
    checklist_id: string;         // FK:
readiness_checklists.checklist_id
    item_number: number;
    category: string;
    description: string;
    validation_method: 'manual' | 'automated' | 'hybrid';
    validation_criteria: jsonb;
    status: 'pending' | 'passed' | 'failed' | 'waived' |
'in_progress';
    validated_at: timestamp;
    validated_by: string;         // FK: staff.staff_id
    validation_evidence: {
        photos: string[];
        sensor_data: jsonb;
        signatures: string[];
        notes: string;
    };
    failure_actions: {
        escalation_path: string[];
        resolution_deadline: timestamp;
        blocking: boolean;        // Blocks go-live if failed
    };
}

```

```

    };
}

interface SystemHealthCheck {
    health_check_id: string;           // PK: uuid
    venue_id: string;                  // FK: venues.venue_id
    system_id: string;                 // FK:
    infrastructure_systems.system_id
    check_timestamp: timestamp;
    check_type: 'connectivity' | 'performance' | 'capacity' |
'failover' | 'integration';
    health_status: 'healthy' | 'degraded' | 'critical' | 'offline';
    health_score: number;              // 0-100
    metrics: {
        response_time_ms: number;
        error_rate: number;
        uptime_percentage: number;
        capacity_used: number;
        capacity_available: number;
        last_failure: timestamp;
    };
    automated: boolean;
    next_check: timestamp;
    alert_threshold: number;
    remediation_required: boolean;
}

interface ReadinessScore {
    score_id: string;                 // PK: uuid
    venue_id: string;                 // FK: venues.venue_id
    event_id: string;                 // FK: events.event_id
    calculated_at: timestamp;
    overall_score: number;            // 0-100
    category_scores: {
        safety_readiness: number;
        technical_readiness: number;
    };
}

```

```

    operational_readiness: number;
    staffing_readiness: number;
    compliance_readiness: number;
};
go_no_go_status: 'go' | 'conditional_go' | 'no_go' | 'pending';
blocking_issues: {
    issue_id: string;
    severity: 'critical' | 'high';
    description: string;
    resolution_required: boolean;
}[];
recommendations: string[];
calculation_metadata: {
    algorithm_version: string;
    confidence_level: number;
    data_completeness: number;
};
}

```

```

interface StaffAssignment {
    assignment_id: string;           // PK: uuid
    event_id: string;               // FK: events.event_id
    staff_id: string;               // FK: staff.staff_id
    role: string;
    zone_assignment: string;         // FK: venue_zones.zone_id
    shift_start: timestamp;
    shift_end: timestamp;
    certification_required: string[];
    certification_verified: boolean;
    check_in_status: 'pending' | 'checked_in' | 'no_show' |
'replaced';
    check_in_time: timestamp;
    replacement_staff_id: string;    // FK: staff.staff_id
    communication_channels: {
        radio_channel: string;
        phone: string;
    };
}

```

```

        team_assignment: string;
    };
}

interface PreEventInspection {
    inspection_id: string;           // PK: uuid
    venue_id: string;               // FK: venues.venue_id
    event_id: string;              // FK: events.event_id
    inspection_type: 'safety' | 'fire_marshall' | 'health' |
'building' | 'insurance';
    inspector_name: string;
    inspector_organization: string;
    scheduled_time: timestamp;
    completed_time: timestamp;
    status: 'scheduled' | 'in_progress' | 'passed' |
'conditional_pass' | 'failed';
    findings: {
        finding_id: string;
        severity: 'critical' | 'major' | 'minor';
        description: string;
        location: string;
        photo_evidence: string[];
        resolution_required: boolean;
        resolved: boolean;
    }[];
    certificate_issued: boolean;
    certificate_number: string;
    valid_until: timestamp;
}

interface ValidationGate {
    gate_id: string;               // PK: uuid
    venue_id: string;             // FK: venues.venue_id
    event_id: string;            // FK: events.event_id
    gate_name: string;           // e.g., "T-7 Safety Gate",
                                "T-1 Systems Gate"
}

```

```

    temporal_marker: string;           // T-7, T-3, T-1, T-0
    required_checks: string[];         // FK:
    checklist_item.item_id[]
    gate_status: 'pending' | 'passed' | 'failed' | 'waived';
    passed_at: timestamp;
    waived_by: string;                 // FK: staff.staff_id
    waiver_reason: string;
    blocking: boolean;                // Blocks progression if
    failed
    approval_required: {
      required: boolean;
      approver_role: string;
      approved_by: string;
      approved_at: timestamp;
    };
  };
}

```

2. API ENDPOINTS

Readiness Management API

typescript

```

TypeScript
// =====
// READINESS CHECKLIST ENDPOINTS
// =====

/**
 * GET
 * /api/v1/venues/:venueId/events/:eventId/readiness/checklists
 *
 * Retrieve all readiness checklists for an event
 */
interface GetChecklistsRequest {

```

```

venueId: string;
eventId: string;
filters?: {
    type?: ChecklistType[];
    status?: ChecklistStatus[];
    phase?: TemporalPhase[];
    assignedTo?: string;
};
}

interface GetChecklistsResponse {
    checklists: ReadinessChecklist[];
    summary: {
        total: number;
        completed: number;
        in_progress: number;
        overdue: number;
        completion_percentage: number;
    };
    critical_items: ChecklistItem[];    // Items blocking go-live
}

/**
 * POST
 * /api/v1/venues/:venueId/events/:eventId/readiness/checklists
 *
 * Create new readiness checklist from template
 */
interface CreateChecklistRequest {
    venueId: string;
    eventId: string;
    template_id?: string;                // Use pre-defined template
    checklist_type: ChecklistType;
    temporal_phase: TemporalPhase;
    assigned_to: string;
    custom_items?: ChecklistItem[];    // Add custom items
}

```

```

}

interface CreateChecklistResponse {
    checklist: ReadinessChecklist;
    items: ChecklistItem[];
}

/**
 * PUT /api/v1/readiness/checklist-items/:itemId/validate
 *
 * Mark checklist item as validated (pass/fail)
 */
interface ValidateItemRequest {
    itemId: string;
    status: 'passed' | 'failed' | 'waived';
    evidence: {
        photos?: File[];
        sensor_data?: jsonb;
        notes: string;
        validated_by: string;
    };
    failure_action?: {
        escalate: boolean;
        assign_remediation_to: string;
        deadline: timestamp;
    };
}

interface ValidateItemResponse {
    item: ChecklistItem;
    checklist_updated: ReadinessChecklist;
    triggers: {
        alerts_sent: string[];
        escalations_created: string[];
        gates_affected: string[];
    };
};

```



```

}

// =====
// SYSTEM HEALTH MONITORING
// =====

/**
 * POST /api/v1/venues/:venueId/systems/health-check
 *
 * Trigger automated health checks across all systems
 */
interface TriggerHealthCheckRequest {
    venueId: string;
    systems?: string[];           // Specific systems, or all
    if empty
    check_types?: HealthCheckType[];
    priority: 'immediate' | 'scheduled';
}

interface TriggerHealthCheckResponse {
    checks_initiated: number;
    estimated_completion: timestamp;
    check_ids: string[];
}

/**
 * GET /api/v1/venues/:venueId/systems/health-status
 *
 * Real-time health status of all venue systems
 */
interface GetHealthStatusRequest {
    venueId: string;
    filters?: {
        health_status?: HealthStatus[];
        system_types?: string[];
        critical_only?: boolean;
    };
}

```

```

    };
}

interface GetHealthStatusResponse {
    timestamp: timestamp;
    overall_health: 'healthy' | 'degraded' | 'critical';
    overall_score: number;
    systems: {
        system_id: string;
        system_name: string;
        health_status: HealthStatus;
        health_score: number;
        last_check: timestamp;
        issues: string[];
    }[];
    degraded_systems: number;
    critical_systems: number;
    alerts_active: number;
}

/**
 * WebSocket: /ws/venues/:venueId/health-monitor
 *
 * Real-time health monitoring stream
 */
interface HealthMonitorStream {
    type: 'health_update' | 'alert' | 'recovery';
    timestamp: timestamp;
    system_id: string;
    previous_status: HealthStatus;
    current_status: HealthStatus;
    metrics: SystemHealthMetrics;
    requires_action: boolean;
}

// =====

```

```

// READINESS SCORING
// =====

/**
 * POST
 * /api/v1/venues/:venueId/events/:eventId/readiness/calculate
 *
 * Calculate current readiness score
 */
interface CalculateReadinessRequest {
  venueId: string;
  eventId: string;
  recalculate_all?: boolean;           // Force full recalculation
}

interface CalculateReadinessResponse {
  score: ReadinessScore;
  breakdown: {
    safety: {
      score: number;
      items_complete: number;
      items_total: number;
      critical_issues: number;
    };
    technical: { /* same structure */ };
    operational: { /* same structure */ };
    staffing: { /* same structure */ };
    compliance: { /* same structure */ };
  };
  go_no_go: {
    status: GoNoGoStatus;
    confidence: number;
    blocking_issues: BlockingIssue[];
    recommendations: string[];
  };
  trend: {

```

```

        previous_score: number;
        change: number;
        projection: number;           // Projected score at T-0
    };
}

/**
 * GET /api/v1/venues/:venueId/events/:eventId/readiness/gates
 *
 * Get validation gate status
 */
interface GetGatesRequest {
    venueId: string;
    eventId: string;
}

interface GetGatesResponse {
    gates: ValidationGate[];
    current_gate: ValidationGate;     // Next gate to pass
    gates_passed: number;
    gates_remaining: number;
    timeline: {
        gate_name: string;
        temporal_marker: string;
        due_date: timestamp;
        status: GateStatus;
    }[];
}

/**
 * POST /api/v1/readiness/gates/:gateId/override
 *
 * Override/waive a validation gate (requires approval)
 */
interface OverrideGateRequest {
    gateId: string;

```

```

    action: 'waive' | 'force_pass';
    reason: string;
    approved_by: string;
    conditions?: string[];           // Conditional pass
    requirements
}

interface OverrideGateResponse {
    gate: ValidationGate;
    audit_entry: {
        action: string;
        timestamp: timestamp;
        approved_by: string;
        reason: string;
    };
    downstream_effects: {
        gates_unlocked: string[];
        checklists_activated: string[];
    };
}

// =====
// STAFF READINESS
// =====

/**
 * POST /api/v1/events/:eventId/staff/assignments
 *
 * Create staff assignments for event
 */
interface CreateAssignmentsRequest {
    eventId: string;
    assignments: {
        staff_id: string;
        role: string;
        zone: string;
    };
}

```

```

        shift_start: timestamp;
        shift_end: timestamp;
    }[];
    auto_verify_certifications?: boolean;
}

interface CreateAssignmentsResponse {
    assignments: StaffAssignment[];
    warnings: {
        staff_id: string;
        warning: string;
        severity: 'critical' | 'warning';
    }[];
    coverage: {
        zone: string;
        required_staff: number;
        assigned_staff: number;
        coverage_percentage: number;
    }[];
}

/**
 * PUT /api/v1/staff/assignments/:assignmentId/check-in
 *
 * Staff check-in for event
 */
interface StaffCheckInRequest {
    assignmentId: string;
    check_in_method: 'badge_scan' | 'mobile_app' | 'manual';
    location: {
        latitude: number;
        longitude: number;
    };
    verification: {
        photo?: string;
        signature?: string;
    };
}

```

```

    };
}

interface StaffCheckInResponse {
    assignment: StaffAssignment;
    briefing_required: boolean;
    equipment_assignment: {
        radio: string;
        tablet: string;
        access_credentials: string[];
    };
    zone_access_granted: string[];
}

/**
 * GET /api/v1/events/:eventId/staff/readiness
 *
 * Staff readiness dashboard
 */
interface GetStaffReadinessResponse {
    event_id: string;
    total_positions: number;
    filled_positions: number;
    checked_in: number;
    no_shows: number;
    coverage_by_zone: {
        zone: string;
        required: number;
        assigned: number;
        checked_in: number;
        coverage_percentage: number;
    }[];
    certification_gaps: {
        staff_id: string;
        staff_name: string;
        role: string;
    };
}

```

```

        missing_certifications: string[];
    }[];
}

// =====
// PRE-EVENT INSPECTIONS
// =====

/**
 * POST /api/v1/venues/:venueId/inspections
 *
 * Schedule pre-event inspection
 */
interface ScheduleInspectionRequest {
    venueId: string;
    eventId: string;
    inspection_type: InspectionType;
    scheduled_time: timestamp;
    inspector: {
        name: string;
        organization: string;
        contact: string;
    };
    areas_to_inspect: string[]; // FK: venue_zones.zone_id[]
}

interface ScheduleInspectionResponse {
    inspection: PreEventInspection;
    notification_sent: boolean;
    preparation_checklist: ChecklistItem[];
}

/**
 * PUT /api/v1/inspections/:inspectionId/findings
 *
 * Record inspection findings

```



```

*/
interface RecordFindingsRequest {
  inspectionId: string;
  findings: {
    severity: 'critical' | 'major' | 'minor';
    description: string;
    location: string;
    photos: File[];
    resolution_required: boolean;
  }[];
  overall_result: 'passed' | 'conditional_pass' | 'failed';
  certificate_details?: {
    certificate_number: string;
    valid_until: timestamp;
  };
}

interface RecordFindingsResponse {
  inspection: PreEventInspection;
  remediation_tasks: {
    task_id: string;
    finding_id: string;
    assigned_to: string;
    deadline: timestamp;
    priority: 'critical' | 'high';
  }[];
  impact: {
    readiness_score_change: number;
    gates_affected: string[];
    go_no_go_impact: boolean;
  };
}

```

3. UI COMPONENTS

Readiness Dashboard

typescript

TypeScript

```
// =====  
// MAIN READINESS DASHBOARD COMPONENT  
// =====  
  
interface ReadinessDashboardProps {  
  venueId: string;  
  eventId: string;  
  userRole: UserRole;  
}  
  
const ReadinessDashboard: React.FC<ReadinessDashboardProps> = ({  
  venueId,  
  eventId,  
  userRole  
}) => {  
  return (  
    <DashboardLayout>  
      {/* Top: Overall Readiness Score */}  
      <ReadinessScoreCard  
        score={readinessScore}  
        goNoGoStatus={goNoGoStatus}  
        trending={scoreTrend}  
      />  
  
      {/* Validation Gates Timeline */}  
      <GatesTimeline  
        gates={validationGates}  
        currentGate={currentGate}  
        allowOverride={userRole === 'admin'}  
      />  
  
      {/* Category Breakdown */}  
      <Grid columns={5}>  
        <CategoryCard
```

```

        title="Safety Readiness"
        score={categoryScores.safety}
        items={safetyItems}
        color="#1B3A4B"
    />
    <CategoryCard
        title="Technical Systems"
        score={categoryScores.technical}
        items={technicalItems}
        color="#3A7B8A"
    />
    {/* Additional categories... */}
</Grid>

{/* Critical Issues Alert */}
{blockingIssues.length > 0 && (
    <CriticalIssuesPanel
        issues={blockingIssues}
        onResolve={handleResolveIssue}
    />
)}

{/* Active Checklists */}
<ChecklistsOverview
    checklists={activeChecklists}
    onSelectChecklist={handleSelectChecklist}
/>

{/* Staff Readiness */}
<StaffReadinessPanel
    staffing={staffingStatus}
    onManageStaff={handleManageStaff}
/>

{/* System Health Monitor */}
<SystemHealthPanel

```

```

        systems={systemHealth}
        alertsActive={healthAlerts}
      />
    </DashboardLayout>
  );
};

// =====
// READINESS SCORE CARD
// =====

interface ReadinessScoreCardProps {
  score: ReadinessScore;
  goNoGoStatus: GoNoGoStatus;
  trending: 'improving' | 'declining' | 'stable';
}

const ReadinessScoreCard: React.FC<ReadinessScoreCardProps> = ({
  score,
  goNoGoStatus,
  trending
}) => {
  const getScoreColor = (score: number) => {
    if (score >= 90) return '#9AAF45'; // Green
    if (score >= 75) return '#F5A623'; // Yellow
    return '#D0021B'; // Red
  };

  return (
    <Card sx={{ background: '#1B3A4B', color: 'white', mb: 3 }}>
      <CardContent>
        <Grid container spacing={3}>
          { /* Overall Score */ }
          <Grid item xs={12} md={4}>
            <Box textAlign="center">

```

```

        <Typography variant="h6">Overall
Readiness</Typography>
        <Box position="relative" display="inline-flex"
mt={2}>
            <CircularProgress
                variant="determinate"
                value={score.overall_score}
                size={120}
                thickness={6}
                sx={{ color: getScoreColor(score.overall_score)
}}
            />
            <Box
                position="absolute"
                top={0}
                left={0}
                bottom={0}
                right={0}
                display="flex"
                alignItems="center"
                justifyContent="center"
            >
                <Typography variant="h3" component="div">
                    {score.overall_score}
                </Typography>
            </Box>
        </Box>
        <TrendIndicator direction={trending} />
    </Box>
</Grid>

    {/* Go/No-Go Status */}
    <Grid item xs={12} md={4}>
        <Box textAlign="center">
            <Typography variant="h6">Go/No-Go
Status</Typography>

```

```

        <Chip
          label={goNoGoStatus.toUpperCase()}
          color={getGoNoGoColor(goNoGoStatus)}
          size="large"
          sx={{ mt: 2, fontSize: '1.2rem', py: 3 }}
        />
        {score.blocking_issues.length > 0 && (
          <Alert severity="error" sx={{ mt: 2 }}>
            {score.blocking_issues.length} blocking
issue(s)
          </Alert>
        )}
      </Box>
    </Grid>

    {/* Category Scores */}
    <Grid item xs={12} md={4}>
      <Typography variant="h6" gutterBottom>
        Category Breakdown
      </Typography>
      <Stack spacing={1}>

{Object.entries(score.category_scores).map(([category, score]) =>
(
      <Box key={category}>
        <Box display="flex"
justifyContent="space-between" mb={0.5}>
          <Typography variant="body2">
            {category.replace('_', ' ')}
          </Typography>
          <Typography variant="body2"
fontWeight="bold">
            {score}%
          </Typography>
        </Box>
        <LinearProgress

```

```

        variant="determinate"
        value={score}
        sx={{
            height: 8,
            borderRadius: 4,
            backgroundColor: 'rgba(255,255,255,0.2)',
            '& .MuiLinearProgress-bar': {
                backgroundColor: getScoreColor(score)
            }
        }}
    />
</Box>
    )))
</Stack>
</Grid>
</Grid>
</CardContent>
</Card>
    );
};

// =====
// VALIDATION GATES TIMELINE
// =====

interface GatesTimelineProps {
    gates: ValidationGate[];
    currentGate: ValidationGate;
    allowOverride: boolean;
}

const GatesTimeline: React.FC<GatesTimelineProps> = ({
    gates,
    currentGate,
    allowOverride
}) => {

```

```

return (
  <Card sx={{ mb: 3 }}>
    <CardHeader title="Validation Gates" />
    <CardContent>
      <Stepper activeStep={getCurrentGateIndex(gates,
currentGate)} alternativeLabel>
        {gates.map((gate) => (
          <Step key={gate.gate_id} completed={gate.gate_status
=== 'passed'}>
            <StepLabel
              error={gate.gate_status === 'failed'}
              StepIconComponent={() => (
                <GateIcon
                  status={gate.gate_status}
                  temporal={gate.temporal_marker}
                  blocking={gate.blocking}
                />
              )}
            >
              <Typography
variant="subtitle2">{gate.gate_name}</Typography>
              <Typography variant="caption"
color="text.secondary">
                {gate.temporal_marker}
              </Typography>
            </StepLabel>
            <StepContent>
              <Box sx={{ mt: 1 }}>
                <Typography variant="body2">
                  {gate.required_checks.length} checks required
                </Typography>
                {gate.gate_status === 'failed' && allowOverride
&& (
                  <Button
                    size="small"
                    color="warning"

```



```

                startIcon={<WarningIcon />}
                onClick={() =>
handleOverrideGate(gate.gate_id)}
            >
                Override Gate
            </Button>
        )}
    </Box>
</StepContent>
</Step>
    )})
</Stepper>
</CardContent>
</Card>
);
};

// =====
// CHECKLIST INTERFACE
// =====

interface ChecklistInterfaceProps {
    checklist: ReadinessChecklist;
    items: ChecklistItem[];
    onValidateItem: (itemId: string, validation: ValidationData) =>
void;
    userPermissions: string[];
}

const ChecklistInterface: React.FC<ChecklistInterfaceProps> = ({
    checklist,
    items,
    onValidateItem,
    userPermissions
}) => {

```

```

    const [selectedItem, setSelectedItem] = useState<ChecklistItem
| null>(null);
    const [validationMode, setValidationMode] = useState(false);

    return (
      <Box>
        {/* Checklist Header */}
        <Card sx={{ mb: 2, background: '#3A7B8A', color: 'white'
}}>
          <CardContent>
            <Grid container spacing={2} alignItems="center">
              <Grid item xs={12} md={6}>
                <Typography
variant="h6">{checklist.checklist_type}</Typography>
                <Typography variant="caption">
                  {checklist.temporal_phase} | Due:
{formatDate(checklist.due_by)}
                </Typography>
              </Grid>
              <Grid item xs={12} md={6} textAlign="right">
                <Typography variant="h4">
                  {checklist.completed_items} /
{checklist.total_items}
                </Typography>
                <Typography variant="body2">
                  {checklist.completion_percentage}% Complete
                </Typography>
                <LinearProgress
                  variant="determinate"
                  value={checklist.completion_percentage}
                  sx={{ mt: 1, height: 8, borderRadius: 4 }}
                />
              </Grid>
            </Grid>
          </CardContent>
        </Card>

```

```

    { /* Items List */ }
    <List>
      {items.map((item, index) => (
        <React.Fragment key={item.item_id}>
          <ListItem
            button
            onClick={() => setSelectedItem(item)}
            sx={{
              border: '1px solid',
              borderColor: getItemBorderColor(item.status),
              borderRadius: 1,
              mb: 1,
              '&:hover': { backgroundColor: '#F4F6F7' }
            }}
          >
            <ListItemIcon>
              <Checkbox
                checked={item.status === 'passed'}
                disabled={item.status === 'waived'}
                color="success"
              />
            </ListItemIcon>
            <ListItemText
              primary={
                <Box display="flex" alignItems="center"
gap={1}>

                  <Typography variant="body1">
                    {index + 1}. {item.description}
                  </Typography>
                  <StatusChip status={item.status} size="small"

                />

                  </Box>
                }
              secondary={
                <Box mt={0.5}>

```

```

                <Typography variant="caption"
color="text.secondary">
                    {item.category} • {item.validation_method}
                </Typography>
                {item.failure_actions?.blocking && (
                    <Chip
                        label="BLOCKING"
                        size="small"
                        color="error"
                        sx={{ ml: 1 }}
                    />
                )}
            </Box>
        }
    />
    <ListItemSecondaryAction>
        <Button
            variant="contained"
            size="small"
            onClick={() => handleStartValidation(item)}
            disabled={!canValidate(item, userPermissions)}
        >
            Validate
        </Button>
    </ListItemSecondaryAction>
</ListItem>
</React.Fragment>
    )})
</List>

{/* Validation Modal */}
{selectedItem && validationMode && (
    <ValidationModal
        item={selectedItem}
        onComplete={onValidateItem}
        onCancel={() => setValidationMode(false)}
    >

```

```

        />
      )}
    </Box>
  );
};

// =====
// SYSTEM HEALTH MONITOR PANEL
// =====

interface SystemHealthPanelProps {
  systems: SystemHealthCheck[];
  alertsActive: number;
}

const SystemHealthPanel: React.FC<SystemHealthPanelProps> = ({
  systems,
  alertsActive
}) => {
  return (
    <Card>
      <CardHeader
        title="System Health Monitor"
        action={
          <Chip
            label={`${alertsActive} Alerts`}
            color="error"
            variant={alertsActive > 0 ? "filled" : "outlined"}
          />
        }
      />
      <CardContent>
        <TableContainer>
          <Table>
            <TableHead>
              <TableRow sx={{ backgroundColor: '#F4F6F7' }}>

```

```

        <TableCell>System</TableCell>
        <TableCell>Status</TableCell>
        <TableCell>Health Score</TableCell>
        <TableCell>Response Time</TableCell>
        <TableCell>Uptime</TableCell>
        <TableCell>Last Check</TableCell>
        <TableCell>Actions</TableCell>
    </TableRow>
</TableHead>
<TableBody>
    {systems.map((system) => (
        <TableRow key={system.health_check_id}>
            <TableCell>{system.system_id}</TableCell>
            <TableCell>
                <StatusIndicator
status={system.health_status} />
            </TableCell>
            <TableCell>
                <Box display="flex" alignItems="center"
gap={1}>
                    <Box sx={{ width: '100px' }}>
                        <LinearProgress
                            variant="determinate"
                            value={system.health_score}

color={getHealthColor(system.health_score)}
                        />
                    </Box>
                    <Typography variant="body2">
                        {system.health_score}
                    </Typography>
                </Box>
            </TableCell>

<TableCell>{system.metrics.response_time_ms}ms</TableCell>

```

```

<TableCell>{system.metrics.uptime_percentage}%</TableCell>

<TableCell>{formatRelativeTime(system.check_timestamp)}</TableCel
l>

        <TableCell>
            {system.remediation_required && (
                <Button size="small" color="error">
                    Remediate
                </Button>
            )}
        </TableCell>
    </TableRow>
    )})
</TableBody>
</Table>
</TableContainer>
</CardContent>
</Card>
);
};

// =====
// STAFF READINESS PANEL
// =====

interface StaffReadinessPanelProps {
    staffing: StaffReadiness;
    onManageStaff: () => void;
}

const StaffReadinessPanel: React.FC<StaffReadinessPanelProps> =
({
    staffing,
    onManageStaff
}) => {

```

```

return (
  <Card sx={{ mb: 3 }}>
    <CardHeader
      title="Staff Readiness"
      action={
        <Button variant="outlined" onClick={onManageStaff}>
          Manage Assignments
        </Button>
      }
    </CardHeader>
    <CardContent>
      <Grid container spacing={3}>
        {/* Overall Stats */}
        <Grid item xs={12} md={3}>
          <StatCard
            title="Total Positions"
            value={staffing.total_positions}
            color="#1B3A4B"
          </StatCard>
        </Grid>
        <Grid item xs={12} md={3}>
          <StatCard
            title="Filled"
            value={staffing.filled_positions}
            percentage={staffing.filled_positions /
staffing.total_positions}
            color="#3A7B8A"
          </StatCard>
        </Grid>
        <Grid item xs={12} md={3}>
          <StatCard
            title="Checked In"
            value={staffing.checked_in}
            percentage={staffing.checked_in /
staffing.filled_positions}
            color="#9AAF45"
          </StatCard>
        </Grid>
      </Grid>
    </CardContent>
  </Card>
)

```



```

    />
</Grid>
<Grid item xs={12} md={3}>
  <StatCard
    title="No Shows"
    value={staffing.no_shows}
    alert={staffing.no_shows > 0}
    color="#D0021B"
  />
</Grid>

{/* Zone Coverage */}
<Grid item xs={12}>
  <Typography variant="h6" gutterBottom>
    Zone Coverage
  </Typography>
  <TableContainer>
    <Table size="small">
      <TableHead>
        <TableRow>
          <TableCell>Zone</TableCell>
          <TableCell>Required</TableCell>
          <TableCell>Assigned</TableCell>
          <TableCell>Checked In</TableCell>
          <TableCell>Coverage</TableCell>
        </TableRow>
      </TableHead>
      <TableBody>
        {staffing.coverage_by_zone.map((zone) => (
          <TableRow key={zone.zone}>
            <TableCell>{zone.zone}</TableCell>
            <TableCell>{zone.required}</TableCell>
            <TableCell>{zone.assigned}</TableCell>
            <TableCell>{zone.checked_in}</TableCell>
            <TableCell>

```

```

        <Box display="flex" alignItems="center"
gap={1}>

        <LinearProgress
            variant="determinate"
            value={zone.coverage_percentage}
            sx={{ flexGrow: 1 }}

color={getCoverageColor(zone.coverage_percentage)}
        />
        <Typography variant="body2">
            {zone.coverage_percentage}%
        </Typography>
    </Box>
</TableCell>
</TableRow>
    )})
</TableBody>
</Table>
</TableContainer>
</Grid>

    { /* Certification Gaps */ }
    { staffing.certification_gaps.length > 0 && (
        <Grid item xs={12}>
            <Alert severity="warning">
                <AlertTitle>Certification Gaps
Detected</AlertTitle>
                { staffing.certification_gaps.map((gap) => (
                    <Typography key={gap.staff_id} variant="body2">
                        {gap.staff_name} ({gap.role}) missing:
{gap.missing_certifications.join(', ')}
                    </Typography>
                ))}
            </Alert>
        </Grid>
    )}

```

```
        </Grid>
      </CardContent>
    </Card>
  );
};
```

4. TEMPORAL SEQUENCE

T-7 to T-0 Day-by-Day Breakdown

typescript

```
TypeScript
// =====
// READINESS TEMPORAL SEQUENCE
// =====

const READY_TEMPORAL_SEQUENCE = {
  "T-7_DAYS": {
    phase: "Initial Validation",
    key_activities: [
      "Comprehensive system health checks",
      "Staff assignment finalization",
      "Initial safety inspections",
      "Compliance documentation review"
    ],
    validation_gates: [
      {
        gate_name: "T-7 Safety Gate",
        required_checks: [
          "Fire suppression systems operational",
          "Emergency lighting tested",
          "Evacuation routes clear",
          "AED devices inspected and charged",
          "First aid stations stocked"
        ]
      }
    ]
  }
};
```

```

        ],
        automated: true,
        blocking: true
    }
],
system_requirements: {
    health_score_minimum: 85,
    critical_systems_operational: "100%",
    degraded_systems_allowed: 2
},
deliverables: [
    "Initial readiness score baseline",
    "Staff assignment roster",
    "System health report",
    "Safety inspection schedule"
]
},

"T-5_DAYS": {
    phase: "Technical Validation",
    key_activities: [
        "Integration testing",
        "Load testing critical systems",
        "Failover testing",
        "Capacity verification",
        "Performance baseline establishment"
    ],
    validation_gates: [
        {
            gate_name: "T-5 Technical Gate",
            required_checks: [
                "All integrations responding < 100ms",
                "Database query performance validated",
                "API rate limits tested",
                "Redundancy systems verified",
                "Backup systems online"
            ]
        }
    ]
}

```

```

        ],
        automated: true,
        blocking: true
    }
],
automated_tests: [
    {
        test_name: "Ticketing System Load Test",
        scenario: "Simulate 50,000 concurrent users",
        pass_criteria: {
            response_time_p95: "< 500ms",
            error_rate: "< 0.1%",
            throughput: "> 10,000 req/sec"
        }
    },
    {
        test_name: "Payment Processing Failover",
        scenario: "Primary gateway failure",
        pass_criteria: {
            failover_time: "< 30 seconds",
            transaction_loss: "0%",
            alert_notification: "< 10 seconds"
        }
    }
]
},

"T-3_DAYS": {
    phase: "Operational Validation",
    key_activities: [
        "Staff training verification",
        "Emergency drill execution",
        "Operational playbook review",
        "Communication system testing",
        "Command center setup"
    ],

```

```
validation_gates: [
  {
    gate_name: "T-3 Operations Gate",
    required_checks: [
      "100% staff certified and trained",
      "Emergency drills completed successfully",
      "Communication channels tested",
      "Command center operational",
      "Incident response times validated"
    ],
    automated: false,
    blocking: true
  }
],
emergency_drills: [
  {
    drill_type: "Evacuation Drill",
    scenario: "Fire alarm in Section 200",
    participants: ["Security", "Operations", "Medical",
"Guest Services"],
    success_criteria: {
      alert_time: "< 30 seconds",
      response_deployment: "< 2 minutes",
      evacuation_time: "< 8 minutes",
      all_clear_confirmation: "< 10 minutes"
    },
    documentation_required: true
  },
  {
    drill_type: "Medical Emergency",
    scenario: "Cardiac arrest in suite level",
    participants: ["Medical", "Security", "Operations"],
    success_criteria: {
      response_time: "< 90 seconds",
      AED_deployment: "< 3 minutes",
      EMS_coordination: "< 5 minutes"
    }
  }
]
```

```

    }
  }
]
},

"T-1_DAY": {
  phase: "Final Validation",
  key_activities: [
    "Final walkthrough",
    "Go/No-Go meeting",
    "Last-minute readiness adjustments",
    "Staffing confirmation",
    "Final system checks"
  ],
  validation_gates: [
    {
      gate_name: "T-1 Final Gate",
      required_checks: [
        "Overall readiness score ≥ 90",
        "Zero critical issues outstanding",
        "All staff checked in or accounted for",
        "All systems green",
        "Weather forecast reviewed",
        "Contingency plans activated if needed"
      ],
      automated: false,
      blocking: true,
      requires_executive_approval: true
    }
  ],
  go_no_go_decision: {
    decision_makers: ["Venue GM", "Operations Director",
"Security Chief"],
    decision_criteria: {
      readiness_score_minimum: 90,
      critical_issues_allowed: 0,

```

```

        high_issues_allowed: 2,
        staff_coverage_minimum: "95%",
        system_health_minimum: "98%"
    },
    contingency_triggers: [
        {
            condition: "Severe weather forecast",
            action: "Activate weather contingency plan",
            approval_required: true
        },
        {
            condition: "Staff shortage > 10%",
            action: "Deploy backup staffing plan",
            approval_required: false
        }
    ]
}
},

"T-4_HOURS": {
    phase: "Pre-Event Setup",
    key_activities: [
        "Final equipment checks",
        "Staff positioning",
        "Command center activation",
        "Communication channel opening",
        "Guest services preparation"
    ],
    automated_checks: [
        "All entry gates operational",
        "Concession POS systems online",
        "Parking systems ready",
        "Security cameras streaming",
        "PA system functional"
    ],
    staff_activities: [

```



```

{
  role: "Operations",
  tasks: [
    "Open command center",
    "Activate monitoring systems",
    "Verify communication with all zones",
    "Confirm staffing in all areas"
  ]
},
{
  role: "Security",
  tasks: [
    "Deploy security personnel",
    "Test metal detectors",
    "Verify camera coverage",
    "Confirm law enforcement coordination"
  ]
},
{
  role: "Guest Services",
  tasks: [
    "Open guest services stations",
    "Activate mobile app services",
    "Verify accessibility services",
    "Confirm concession readiness"
  ]
}
]
},

"T-1_HOUR": {
  phase: "Gates Open Preparation",
  key_activities: [
    "Final safety sweep",
    "Gates open briefing",
    "Live monitoring activation",

```

```
        "Emergency protocols on standby"
    ],
    final_briefing: {
        attendees: ["All department heads", "Command center
staff"],
        topics: [
            "Weather update",
            "Expected attendance",
            "Any last-minute changes",
            "Emergency contact verification",
            "Go-live confirmation"
        ],
        duration: "15 minutes",
        documentation: true
    }
},

"T-0_GATES_OPEN": {
    phase: "Event Activation",
    transition: "READY → RUN",
    key_activities: [
        "Gates open",
        "Live monitoring begins",
        "Incident tracking active",
        "Real-time orchestration engaged"
    ],
    system_state_changes: {
        monitoring_mode: "active",
        alert_sensitivity: "high",
        response_protocols: "armed",
        evidence_capture: "recording"
    }
}
};
```

5. STATE MACHINE

READY Zone State Transitions

typescript

TypeScript

```
// =====  
// READY ZONE STATE MACHINE  
// =====  
  
type ReadyState =  
  | 'INITIAL_VALIDATION'           // T-7 days  
  | 'TECHNICAL_VALIDATION'        // T-5 days  
  | 'OPERATIONAL_VALIDATION'      // T-3 days  
  | 'FINAL_VALIDATION'            // T-1 day  
  | 'PRE_EVENT_SETUP'             // T-4 hours  
  | 'GATES_OPEN_PREP'             // T-1 hour  
  | 'READY_COMPLETE';            // T-0  
  
interface ReadyStateMachine {  
  current_state: ReadyState;  
  venue_id: string;  
  event_id: string;  
  readiness_score: number;  
  blocking_issues: number;  
  gates_passed: ValidationGate[];  
  gates_pending: ValidationGate[];  
  
  // State transition logic  
  can_transition_to(target_state: ReadyState): {  
    allowed: boolean;  
    reason?: string;  
    required_actions?: string[];  
  };  
  
  transition_to(target_state: ReadyState): {  
    success: boolean;  
    previous_state: ReadyState;  
  };  
};
```

```

        new_state: ReadyState;
        timestamp: Date;
        triggered_actions: string[];
    };
}

// =====
// STATE TRANSITION RULES
// =====

const READY_STATE_TRANSITIONS: Record<ReadyState, {
    next_states: ReadyState[];
    entry_conditions: Condition[];
    on_entry: Action[];
    on_exit: Action[];
}> = {

    INITIAL_VALIDATION: {
        next_states: ['TECHNICAL_VALIDATION'],
        entry_conditions: [
            {
                condition: 'Event scheduled',
                validation: (context) => context.event_date !== null
            },
            {
                condition: 'Blueprint complete',
                validation: (context) => context.blueprint_status ===
'finalized'
            }
        ],
        on_entry: [
            'Activate T-7 validation gate',
            'Schedule safety inspections',
            'Assign staff to checklists',
            'Initialize system health monitoring'
        ],

```

```

    on_exit: [
        'Verify T-7 gate passed',
        'Generate initial readiness report'
    ]
},

TECHNICAL_VALIDATION: {
    next_states: ['OPERATIONAL_VALIDATION'],
    entry_conditions: [
        {
            condition: 'T-7 gate passed',
            validation: (context) =>
context.gates_passed.includes('T-7')
        },
        {
            condition: 'All systems responding',
            validation: (context) => context.system_health_score >=
85
        }
    ],
    on_entry: [
        'Activate T-5 validation gate',
        'Schedule load testing',
        'Initiate failover tests',
        'Verify integration health'
    ],
    on_exit: [
        'Verify T-5 gate passed',
        'Confirm system performance baselines'
    ]
},

OPERATIONAL_VALIDATION: {
    next_states: ['FINAL_VALIDATION'],
    entry_conditions: [
        {

```

```

        condition: 'T-5 gate passed',
        validation: (context) =>
context.gates_passed.includes('T-5')
    },
    {
        condition: 'Staff assignments finalized',
        validation: (context) => context.staff_coverage >= 95
    }
],
on_entry: [
    'Activate T-3 validation gate',
    'Schedule emergency drills',
    'Verify staff certifications',
    'Test communication systems'
],
on_exit: [
    'Verify T-3 gate passed',
    'Confirm drill success',
    'Validate operational readiness'
]
},

FINAL_VALIDATION: {
    next_states: ['PRE_EVENT_SETUP'],
    entry_conditions: [
        {
            condition: 'T-3 gate passed',
            validation: (context) =>
context.gates_passed.includes('T-3')
        },
        {
            condition: 'Readiness score ≥ 90',
            validation: (context) => context.readiness_score >= 90
        },
        {
            condition: 'Zero critical issues',

```

```

        validation: (context) => context.critical_issues === 0
    }
],
on_entry: [
    'Activate T-1 validation gate',
    'Schedule go/no-go meeting',
    'Execute final walkthrough',
    'Prepare contingency plans'
],
on_exit: [
    'Go/No-Go decision made',
    'Executive approval received',
    'Final readiness report generated'
]
},

PRE_EVENT_SETUP: {
    next_states: ['GATES_OPEN_PREP'],
    entry_conditions: [
        {
            condition: 'T-1 gate passed',
            validation: (context) =>
context.gates_passed.includes('T-1')
        },
        {
            condition: 'Go decision confirmed',
            validation: (context) => context.go_no_go === 'GO'
        }
    ],
    on_entry: [
        'Activate command center',
        'Deploy staff to positions',
        'Enable live monitoring',
        'Arm emergency protocols'
    ],
    on_exit: [

```

```

        'Confirm all stations manned',
        'Verify systems live'
    ]
},

GATES_OPEN_PREP: {
    next_states: ['READY_COMPLETE'],
    entry_conditions: [
        {
            condition: 'T-1 hour checkpoint',
            validation: (context) => context.time_to_gates <= 3600
        },
        {
            condition: 'All staff checked in',
            validation: (context) => context.staff_check_in_rate >=
98
        }
    ],
    on_entry: [
        'Execute final safety sweep',
        'Conduct gates-open briefing',
        'Activate guest entry systems',
        'Enable real-time tracking'
    ],
    on_exit: [
        'Open gates',
        'Transition to RUN zone'
    ]
},

READY_COMPLETE: {
    next_states: [], // Terminal state - transitions to RUN zone
    entry_conditions: [
        {
            condition: 'Gates opened',
            validation: (context) => context.gates_status === 'open'

```



```

        }
    ],
    on_entry: [
        'Log transition to RUN',
        'Archive readiness data',
        'Begin live operations tracking'
    ],
    on_exit: []
}
};

// =====
// STATE TRANSITION IMPLEMENTATION
// =====

class ReadyZoneStateMachine {
    private state: ReadyState;
    private context: ReadyContext;

    constructor(initialState: ReadyState, context: ReadyContext) {
        this.state = initialState;
        this.context = context;
    }

    canTransition(targetState: ReadyState): TransitionValidation {
        const currentStateConfig =
READY_STATE_TRANSITIONS[this.state];

        // Check if target state is allowed from current state
        if (!currentStateConfig.next_states.includes(targetState)) {
            return {
                allowed: false,
                reason: `Cannot transition from ${this.state} to
${targetState}`
            };
        }
    }
}

```

```
// Validate all entry conditions for target state
const targetStateConfig =
READY_STATE_TRANSITIONS[targetState];
const failedConditions = targetStateConfig.entry_conditions
  .filter(condition => !condition.validation(this.context))
  .map(condition => condition.condition);

if (failedConditions.length > 0) {
  return {
    allowed: false,
    reason: 'Entry conditions not met',
    required_actions: failedConditions
  };
}

return { allowed: true };
}

transition(targetState: ReadyState): TransitionResult {
  const validation = this.canTransition(targetState);

  if (!validation.allowed) {
    return {
      success: false,
      error: validation.reason,
      required_actions: validation.required_actions
    };
  }

  // Execute exit actions from current state
  const currentStateConfig =
READY_STATE_TRANSITIONS[this.state];
  const exitActions =
this.executeActions(currentStateConfig.on_exit);
}
```

```

    // Update state
    const previousState = this.state;
    this.state = targetState;

    // Execute entry actions for new state
    const targetStateConfig =
READY_STATE_TRANSITIONS[targetState];
    const entryActions =
this.executeActions(targetStateConfig.on_entry);

    // Log transition
    this.logTransition(previousState, targetState);

    return {
        success: true,
        previous_state: previousState,
        new_state: targetState,
        timestamp: new Date(),
        triggered_actions: [...exitActions, ...entryActions]
    };
}

private executeActions(actions: string[]): string[] {
    // Implementation would trigger actual system actions
    return actions;
}

private logTransition(from: ReadyState, to: ReadyState): void {
    console.log(`[READY STATE MACHINE] Transition: ${from} →
${to}`);
    // Log to evidence ledger
}
}

```